DIVANEX TECHNOLOGIES

Addendum Audit

Every listed item marked against the current codebase

The addendum asks for this before any development: each requirement marked Working, Partial, Missing or Not Applicable, with evidence and a proposed action, so that nothing already built is rebuilt.

Audits	**Missing Features Addendum, 16 October 2026**
Codebase	**main @ af976a2**
Stack confirmed	**Next.js 16 App Router, TypeScript, Supabase**
Prepared	**16 October 2026**

How this was checked

Every status below came from reading the code or driving the running site, not from the handbook. Where a claim could be tested against the live site it was, and three findings in this report contradict what the handbook says — those are corrected in Part 4.

WORKING built and verified **PARTIAL** exists but short of the requirement **MISSING** no implementation **N/A** does not apply here

> **Headline: the addendum is right about the big items, and wrong about one.**
> Six of the seven "not built yet" features are genuinely absent. Revision rollback is the exception — a working whole-content backup and restore already exists; what is missing is per-record version history.

The audit also surfaced four defects the addendum does not mention, found while checking the items it does. They are in Part 3 and matter more than some of the P2 work.

PART 1

Section 2 — the seven "not built yet" features

FEATURE	STATUS	EVIDENCE	PROPOSED ACTION
Page draft & preview	MISSING	No draft field on page or section records. Saving calls `purgeTag` and the change is live (`contentStore.ts:152`). No preview surface exists for pages.	Add a draft payload alongside the live one per collection, a preview route that reads the draft behind admin auth, and separate Save Draft from Publish.
Revision rollback	PARTIAL	No version table. But a full export and a confirmed import exist (`AdminContentToolsView.tsx:109,128`), and the audit trail stores before/after per field (`auditStore.ts:130`).	Keep the backup. Add a snapshot row per save and a compare-and-restore screen for a single record, which is the part that is actually missing.
Media library	MISSING	No upload endpoint, no Supabase Storage call, no multipart handler anywhere in `src/`. Images are pasted URLs.	A Storage bucket, a server-side signed upload, type and size validation, and a picker that stores alt text with the file.
Redirect manager	MISSING	No `redirects()` in `next.config.ts`, no redirect table or store.	A redirects collection read by the existing proxy, with loop detection and a warning when an address in use is changed.
Location pages	MISSING	No route for any of the nine markets under `src/app/`.	One template plus a CMS collection. Blocked on genuine local detail per market — see the note at the end of this part.
Scheduled publishing	MISSING	No scheduled or publish-at field on posts (`blogData.ts`), and nothing in the blog editor. The addendum's "may already be supported" is not the case.	A publish-at timestamp plus a scheduled job. Needs a decision on timezone and on what happens when a run is missed.
SEO validation warnings	MISSING	No validation of any kind in `AdminSeoView.tsx`. Length, duplicates and missing fields are unchecked.	Checks computed on read and shown next to the field. Cheapest item on this list and the one with the most day-to-day value.

> **Location pages need content before they need code.**
> The template is perhaps two days. The nine markets are the real constraint: the addendum itself forbids thin duplicates, and only Jaipur has a real address. Decide what is genuinely true for each market before the work is scheduled, or it will produce nine pages that should not be indexed.

PART 2

Section 3 — requirements to verify

3.1 Page content and section builder

REQUIREMENT	STATUS	FINDING
Edit section fields	WORKING	Every section on every page is reachable; the editor lists the union of the registry and generated defaults so a section cannot be unreachable (`AdminSectionsView.tsx:66`).
Add, duplicate, reorder, hide, remove sections	MISSING	The editor edits sections that exist. There is no add, duplicate, reorder or hide. This is the largest single gap in the addendum and is understated by it.
Desktop / tablet / mobile preview	MISSING	Only the forms screen has a preview iframe (`AdminFormsView.tsx:372`). No page preview.
Page draft / publish / unpublish	MISSING	As Part 1.
Defined fields per section type	WORKING	Sections are typed as heading, items and call-to-action, generated into `SECTION_DEFAULTS`.
No arbitrary HTML or JavaScript	WORKING	Rich text escapes first and then applies markup (`richText.ts:77`), so a tag entered in a field renders as text.

3.2 Dynamic content types

REQUIREMENT	STATUS	FINDING
Creating a service generates its public page	WORKING	24 services live, each with its own address and metadata.
Portfolio linked to case study by a stable relationship	PARTIAL	Two collections joined by matching the project id by hand. This produced 39 live links to pages nobody had written; the links are now suppressed and the console warns, but the relationship is still manual. The addendum is right to call this out.
Status, metadata, archive for services and case studies	MISSING	Only blog posts carry a status. A service or case study is either present and live, or deleted.
Reusable FAQ manager assignable to pages	PARTIAL	FAQs exist inside page and service records; they cannot be written once and attached to several pages.
Testimonials with attribution	PARTIAL	53 records with name, company, role and project. No field records whether permission to publish was given.

3.3 SEO and URL controls

REQUIREMENT	STATUS	FINDING
Per-page title, description, social image, robots	WORKING	16 routes editable (`siteSettings.ts`).
Per-page canonical override	PARTIAL	Canonicals are correct and automatic from the path (`seo.ts:42`) but cannot be overridden.
Sitemap excludes drafts and noindex pages	PARTIAL	Draft posts are excluded (`sitemapEntries.ts:151`). **The per-page noindex flag is not consulted at all** — see Part 3.

robots.txt uses the production domain	**WORKING**	Verified live: host and sitemap both `www.divanextechnologies.com`.
Broken-link check	**WORKING**	Backup, Audit & Search finds links pointing at pages that no longer exist. Confirmed independently: all 109 sitemap addresses return 200, and no internal link 404s.
Structured data from visible information	**WORKING**	Built in `seo.ts` from the same records the page renders.
Warn before a URL change or deletion	**PARTIAL**	Deleting a page warns and names the address that will start returning Not Found (`RecordPagesEditor.tsx:162`). Renaming does not warn, and nothing checks what links in.

3.4 Security, roles and audit

REQUIREMENT	STATUS	FINDING
Permissions enforced server-side	**WORKING**	Every admin route gates on the server. Swept the whole API tree: the only handlers without a gate are sign-in, sign-out, session and the public blog read, which is correct.
Record who changed content, with before and after	**PARTIAL**	Everything through the content store is recorded with field diffs (`content/[collection]/route.ts:108`). **Blog posts, client sprints and subscriber deletions are not recorded at all** — see Part 3.
Drafts and admin APIs not publicly reachable	**WORKING**	No drafts exist to expose. Unauthenticated calls to admin endpoints return 401.
Uploads validated	**N/A**	No upload path exists yet. Becomes a requirement with the media library.
Service-role secret kept off the browser	**WORKING**	`supabase.ts` is marked `server-only`, so an accidental client import fails the build. RLS is on with no permissive policies.
Backup, migration and restore documented	**PARTIAL**	Scripts exist (`scripts/setup-db.js`, `migrate-db.js`) and content backup and restore work. The recovery procedure has not been written down or rehearsed.

3.5 Integrations and dashboard

REQUIREMENT	STATUS	FINDING
Analytics connected to a real source	**WORKING**	Fixed during this audit cycle. The tag was present but refused by the site's own security policy, so the property had been receiving nothing. Now verified reporting from the live site.
Preserve GA4 conversion events	**N/A**	No custom events or conversions are defined in the code. Nothing to preserve; worth configuring deliberately.
Publishing revalidates affected pages	**WORKING**	Each save purges its cache tag and the routes that depend on it (`cache.ts:97`).
Real metrics, no invented figures	**WORKING**	Rebuilt during this audit cycle. The screen previously showed a fixed revenue curve, a 99.9% SLA and a cache hit ratio for a cache the site does not run.
Status and error reporting	**WORKING**	A banner on every admin screen when the database is unreachable; a mail-server check on the forms screen.
Forms, newsletter, chat, email	**WORKING**	Six enquiries stored and retrievable; subscriber list and chat screens operational.

PART 3

Defects the addendum does not list

Found while checking the items above. Each is small to fix and two of them are losing information today.

1. Blog posts leave no audit record

Posts live in their own table rather than the content store, and that path never calls the audit recorder (`blogStore.ts`, `api/blog/route.ts`). Anyone with content rights can publish, rewrite or delete an article and the trail will not show it. The same is true of client sprints and subscriber deletions.

The action names `project.created`, `subscriber.deleted` and `chat.deleted` are declared in the audit type and never written by anything — the intent was there and the calls were never added.

Action: record the same diffs the content store records. Half a day, and it closes a real hole in section 3.4.

2. The sitemap ignores the noindex flag

Draft posts are correctly excluded, but the per-page noindex setting is never read when the sitemap is built (`sitemapEntries.ts`). Setting a page to noindex in the console would leave it listed in the sitemap — telling search engines to crawl a page that then asks not to be indexed.

No page is currently set to noindex, so nothing is wrong on the live site today. It will go wrong the first time the feature is used, which is the worst way to find out.

Action: filter on the flag. An hour.

3. Rate limiting is per-instance and in memory

The limit is 200 page requests per minute per address, counted in the memory of whichever serverless instance answers (`proxy.ts:153`). On a platform that runs many instances the real limit is neither 200 nor predictable, which makes it weaker protection than it looks. It can also catch several genuine visitors sharing one office or mobile network.

Action: move the counter to the database so one limit is actually enforced, and reconsider the number. This is a security control, so it should be a deliberate decision rather than a quiet change.

4. Two components are built and never rendered

`EngagementModels.tsx` and `SecurityComplianceSection.tsx` are complete and unreferenced. They ship in the repository and confuse anyone searching for where something is rendered.

Action: delete them, or wire them in if they were meant to be live.

PART 4

Corrections to the handbook

The addendum was written from the handbook, so where the handbook was wrong the addendum inherits it. Three corrections, and the handbook itself will be reissued.

HANDBOOK SAYS	ACTUALLY
"There is no rollback."	A full content backup and restore exists and works — export to a file, import it back with a confirmation. What is missing is per-record version history, which is a narrower gap and a smaller job.
"Nothing flags a broken link."	Backup, Audit & Search does find links pointing at pages that no longer exist. The missing checks are the SEO ones: duplicate addresses, over-long descriptions, missing image descriptions.
Audit trail covers content changes.	True for everything except blog posts, client sprints and subscriber deletions. See Part 3.

PART 5

Tally and recommended order

STATUS	COUNT	ITEMS
WORKING	18	Section editing, typed fields, HTML escaping, service pages, per-page SEO, robots domain, broken-link check, structured data, server-side permissions, secret handling, analytics, revalidation, real metrics, status reporting, forms, newsletter, chat, draft exposure.
PARTIAL	8	Revision restore, portfolio relationship, FAQ reuse, testimonial permission, canonical override, sitemap noindex, URL-change warnings, content audit coverage, recovery documentation.
MISSING	10	Page draft and preview, media library, redirects, location pages, scheduled publishing, SEO warnings, section add/reorder/hide, responsive preview, page unpublish, service and case status.
N/A	2	Upload validation, GA4 conversion preservation.

Suggested order, and why it differs from the addendum

The addendum puts draft, media and restore first. Two changes are worth considering.

ORDER	WORK	REASON
1	The four defects in Part 3	Not in the addendum, all small, and two are losing information or will mislead search engines the first time a feature is used.
2	SEO validation warnings	P1 in the addendum, the cheapest item on the list, and it prevents mistakes rather than recording them.
3	Draft, preview and publish	The largest safety gain. Everything published today goes live on the first keystroke after Save.
4	Section add, duplicate, reorder, hide	Understated in the addendum. Without it a page's shape still needs a developer, which is most of what a CMS is for.
5	Media library	Needs a storage bucket and an upload path; worth doing properly once rather than quickly twice.
6	Per-record restore, redirects	Redirects matter more once addresses start changing, which the draft and section work will cause.
7	Scheduled publishing, location pages	Location pages are blocked on real market content, not on code.

> **On the acceptance criteria.**
> Of the ten in section 5 of the addendum, two can be demonstrated today: protected operations are refused to unauthorised users, and the existing forms, newsletter, chat, analytics and navigation all work. The other eight depend on the work above.